UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET
Katedra za računarstvo

Blokčein tehnologije

dGameBet: Decentralizovana platforma za klađenje na fudbalske utakmice zasnovana na Ethereum blockchain-u

Student: Andrija Petrović 1751

Mentori:
Prof. dr Dragan Janković
Prof. dr Vladimir Ćirić
Doc. dr Aleksandar Stanimirović
Dipl. inž. Nikola Davidović

SAŽETAK

Ovaj rad predstavlja dGameBet, decentralizovanu aplikaciju (DApp) za klađenje na ishode fudbalskih utakmica izgrađenu na Ethereum blockchain-u. Platforma koristi pametne ugovore za transparentno i automatsko upravljanje sredstvima bez posrednika. Primenjen je Factory dizajn obrazac gde centralni ugovor kreira zasebne ugovore za svaki meč, čime se postiže izolacija sredstava i nezavisnost mečeva.

Rad analizira arhitekturu pametnih ugovora sa više bezbedonosnih obrazaca za zaštitu sredstava korisnika, kao i strategiju testiranja koja obuhvata unit testove, fuzz testove sa matematičkim invarijantama i bezbednosne testove sa napadačkim ugovorima. Backend koristi event-driven arhitekturu sa blockchain indekserom, a frontend je izgrađen sa Next.js 14 i wagmi bibliotekom. Celokupno okruženje je kontejnerizovano kroz Docker Compose.

ABSTRACT

This paper presents dGameBet, a decentralized application (DApp) for betting on football match outcomes built on the Ethereum blockchain. The platform uses smart contracts for transparent and automatic fund management without intermediaries. A Factory design pattern is employed where a central contract creates separate contracts for each match, achieving fund isolation and match independence.

The paper analyzes the smart contract architecture with multiple security patterns for protecting user funds, as well as a testing strategy that includes unit tests, fuzz tests with mathematical invariants, and security tests with attacker contracts. The backend uses an event-driven architecture with a blockchain indexer, and the frontend is built with Next.js 14 and the wagmi library. The entire environment is containerized through Docker Compose.

SADRŽAJ

SAŽETAK 2
ABSTRACT 3
1. UVOD 6
1.1. Kontekst i motivacija 6
1.2. Ciljevi rada 6
1.3. Opseg projekta 6
1.4. Struktura rada 6
2. PREGLED LITERATURE I TEORIJSKI OKVIR 7
2.1. Blockchain tehnologija 7
2.2. Pametni ugovori i Solidity 7
2.3. Decentralizovane aplikacije (DApps) 7
2.4. Postojeća rešenja za decentralizovano klađenje 7
2.5. Bezbedonosni izazovi pametnih ugovora 7
3. METODOLOGIJA 8
3.1. Razvojni pristup 8
3.2. Tehnološki stek 8
3.3. Alati za testiranje 8
3.4. Docker infrastruktura 8
4. ARHITEKTURA PAMETNIH UGOVORA 10
4.1. Pregled arhitekture – Factory dizajn obrazac 10
4.2. BetFactory ugovor 10
4.3. BetMatch ugovor 12
4.3.1. Modifikatori za kontrolu pristupa 12
4.3.2. Postavljanje opklade – placeBet() 13
4.3.3. Postavljanje rezultata – setResult() 13
4.3.4. Isplata nagrada – claimPrize() i Pull Payment Pattern 14
4.3.5. Refund za nerešene mečeve – claimRefund() 14
4.3.6. Dijagram stanja meča 15
4.3.7. Eventi i custom errori 15
4.4. BetMath biblioteka 15
4.5. Bezbedonosni obrasci 16
4.5.1. ReentrancyGuard 16
4.5.2. Pull Payment Pattern 16
4.5.3. Checks-Effects-Interactions (CEI) 16
4.5.4. Custom Errors 16

4.5.5. Access Control ... 16
4.5.6. Input Validation ... 17
5. TESTIRANJE I VERIFIKACIJA ... 18
5.1. Strategija testiranja ... 18
5.2. Unit testovi ... 18
5.3. Fuzz testovi i invarijante ... 19
5.4. Integracioni testovi ... 19
5.5. Bezbedonosni testovi ... 20
6. BACKEND ARHITEKTURA ... 21
6.1. Fastify REST API ... 21
6.2. Blockchain Event Indexer ... 21
6.3. Baza podataka ... 22
6.4. Redis keš i Pub/Sub ... 22
7. FRONTEND APLIKACIJA ... 23
7.1. Next.js 14 aplikacija ... 23
7.2. Integracija sa blockchain-om ... 23
7.3. Korisničko iskustvo – demonstracija ... 24
7.3.1. Kreiranje meča i klađenje ... 24
7.3.2. Rezultati i Pull Payment u praksi ... 25
7.3.3. Portfolio i analitika ... 26
8. ZAKLJUČAK I BUDUĆI RAD ... 27
8.1. Ostvareni rezultati ... 27
8.2. Ograničenja ... 27
8.3. Primena u realnom svetu ... 27
8.3.1. Decentralizovano sportsko klađenje ... 27
8.3.2. Peer-to-peer klađenje bez posrednika ... 27
8.3.3. DAO-upravljane kladioničke zajednice ... 28
8.3.4. Transparentnost i zaštita korisnika ... 28
8.3.5. Integracija sa DeFi ekosistemom ... 28
8.3.6. Regulatorna usklađenost i revizija ... 28
8.4. Budući pravci razvoja ... 29
LITERATURA ... 30

1. UVOD

1.1. Kontekst i motivacija

Tradicionalne onlajn platforme za klađenje pate od fundamentalnih problema poverenja. Korisnici ne mogu da verifikuju kako se upravlja njihovim sredstvima, kako se izračunavaju koeficijenti, niti da li platforma radi fer. Centralizovane platforme mogu da manipulišu rezultatima, zadržavaju dobitke, nameću visoke provizije (10–20%) i proizvoljno ograničavaju pristup korisnicima [1].

Blockchain tehnologija nudi fundamentalnu promenu paradigme omogućavajući aplikacije bez potrebe za poverenjem (trustless), transparentne i decentralizovane. Pametni ugovori – samoizvršavajući programi raspoređeni na blockchain-u – mogu automatizovati logiku klađenja, obezbeđujući fer distribuciju nagrada bez posrednika [2][3].

1.2. Ciljevi rada

Osnovni cilj ovog rada je projektovanje i implementacija sistema pametnih ugovora za decentralizovano klađenje na fudbalske utakmice koristeći Ethereum blockchain. Specifični ciljevi uključuju:

- Projektovati i implementirati sistem pametnih ugovora koristeći Factory dizajn obrazac
- Primeniti bezbednosne obrazce za zaštitu sredstava korisnika
- Ostvariti 100% pokrivenost koda testovima za pametne ugovore, uključujući fuzz testove sa matematičkim invarijantama
- Izgraditi moderan web interfejs sa MetaMask integracijom za direktnu interakciju sa blockchain-om
- Razviti backend indekser za brz pristup podacima sa blockchain-a
- Kontejnerizovati celokupno okruženje kroz Docker Compose

1.3. Opseg projekta

Projekat dGameBet je implementiran kao potpuno funkcionalna aplikacija na Ethereum Sepolia testnet-u. Podržava kompletan tok klađenja: kreiranje mečeva, postavljanje opklada, postavljanje rezultata, isplatu nagrada i refundiranje u slučaju nerešenog rezultata. Sistem ocenjivanja organizatora obezbeđuje reputacioni mehanizam.

1.4. Struktura rada

Rad je organizovan u osam poglavlja. Nakon uvoda, Poglavlje 2 daje pregled relevantne literature i teorijski okvir. Poglavlje 3 opisuje metodologiju, tehnološki stek i Docker infrastrukturu. Poglavlja 4 i 5 predstavljaju jezgro rada – detaljnu analizu pametnih ugovora i strategije testiranja. Poglavlja 6 i 7 pokrivaju backend i frontend arhitekturu, dok Poglavlje 8 daje zaključak i pravce budućeg razvoja.

2. PREGLED LITERATURE I TEORIJSKI OKVIR

2.1. Blockchain tehnologija

Blockchain je distribuirana baza podataka koja čuva zapise u kriptografski povezanim blokovima. Nakamoto [3] je 2008. godine predstavio koncept u Bitcoin whitepaper-u, demonstrirajući peer-to-peer elektronski novčani sistem bez centralnog autoriteta. Ethereum, koji je predložio Buterin [1] 2014. godine, proširio je ovaj koncept uvođenjem Turing-kompletnog programskog jezika, omogućavajući razvoj decentralizovanih aplikacija (DApps).

Ethereum koristi proof-of-stake konsenzus mehanizam (od nadogradnje The Merge u septembru 2022.) i Ethereum Virtual Machine (EVM) za izvršavanje pametnih ugovora. Svaka operacija na EVM-u ima gas trošak, što sprečava beskonačne petlje i obezbeđuje ekonomsku ravnotežu mreže [4].

2.2. Pametni ugovori i Solidity

Pametni ugovori su samoizvršavajući programi čiji se uslovi sporazuma direktno zapisuju u kod. Koncept je prvi predložio Szabo [2] 1997. godine. U Ethereum ekosistemu, pametni ugovori se pišu u Solidity programskom jeziku – statički tipiziranom jeziku visokog nivoa dizajniranom za EVM. Nakon kompajliranja u bytecode, ugovori se raspoređuju na blockchain gde postaju nepromenljivi i javno verifikovani [5].

2.3. Decentralizovane aplikacije (DApps)

DApps su aplikacije čija se backend logika izvršava na decentralizovanom peer-to-peer sistemu umesto na centralizovanim serverima. Tipična arhitektura DApp-a uključuje: pametne ugovore na blockchain-u za poslovnu logiku, web frontend za korisničko iskustvo, i opcionalni backend za indeksiranje podataka sa blockchain-a radi poboljšanja performansi [5].

2.4. Postojeća rešenja za decentralizovano klađenje

Augur je jedan od prvih decentralizovanih prediction market-a koji koristi Ethereum za kreiranje tržišta za predviđanje ishoda događaja. Polymarket je modernija platforma koja koristi Polygon za niže transakcione troškove. Azuro Protocol pruža infrastrukturu za decentralizovano klađenje sa likvidnosnim pool-ovima. dGameBet se razlikuje fokusiranjem na jednostavnost, fiksne iznose opklada i Factory dizajn obrazac za izolaciju mečeva.

2.5. Bezbedonosni izazovi pametnih ugovora

Atzei et al. [6] sistematizovali su napade na Ethereum pametne ugovore, identifikujući reentrancy napade, integer overflow/underflow, i probleme sa kontrolom pristupa kao najčešće ranjivosti. DAO hack iz 2016. godine, gde je napadač iskoristio reentrancy ranjivost za krađu 3.6 miliona ETH-a, ostaje najpoznatiji primer. OpenZeppelin [7] pruža biblioteku proverenih implementacija zaštitnih obrazaca, uključujući ReentrancyGuard koji se koristi u ovom projektu. Consensys [8] je objavio vodič za najbolje prakse bezbednosti pametnih ugovora.

3. METODOLOGIJA

3.1. Razvojni pristup

Projekat je razvijen koristeći Docker-first pristup – svi alati za razvoj, testiranje i pokretanje aplikacije se izvršavaju unutar Docker kontejnera. Ovo obezbeđuje konzistentno okruženje nezavisno od operativnog sistema razvijača i eliminiše probleme tipa „works on my machine“. Iterativni razvoj je pratio tok: pametni ugovori → testovi → backend → frontend.

3.2. Tehnološki stek

Tabela 1. Tehnološki stek projekta dGameBet

Komponenta	Tehnologija	Verzija	Namena
Pametni ugovori	Solidity	0.8.24	On-chain logika klađenja
Framework za ugovore	Foundry	Najnovija	Kompajliranje, testiranje, deploy
Backend server	Fastify + Node.js	20 LTS	REST API, WebSocket, indekser
Frontend	Next.js	14	SSR, React, wagmi integracija
Baza podataka	PostgreSQL	16	Trajno čuvanje indeksiranih podataka
Keš	Redis	7	Keširanje, Pub/Sub poruke
ORM	Drizzle	Najnovija	Type-safe upiti nad PostgreSQL
Blockchain klijent	viem	Najnovija	Čitanje/pisanje na Ethereum
Wallet integracija	wagmi + RainbowKit	Najnovija	MetaMask, WalletConnect
Kontejnerizacija	Docker Compose	Najnovija	Orkestracija svih servisa

3.3. Alati za testiranje

Za testiranje pametnih ugovora koristi se Foundry framework [9] koji pruža forge alat za pokretanje testova napisanih u Solidity-ju. Foundry podržava cheatcode-ove (vm.prank, vm.deal, vm.warp, vm.expectRevert, vm.expectEmit) za simulaciju različitih uslova na blockchain-u. Za backend i frontend testove koristi se Vitest, dok se Playwright koristi za end-to-end testove.

3.4. Docker infrastruktura

Celokupno razvojno i produkcijsko okruženje je kontejnerizovano kroz Docker Compose – nikakvi paketi se ne instaliraju lokalno na mašini, već se sve izvršava unutar kontejnera.

Tabela 2. Docker Compose servisi

Servis	Docker slika	Port	Namena
postgres	postgres:16	5432	Baza podataka sa trajnim podacima
redis	redis:7	6379	Keš i Pub/Sub poruke
backend	Custom (Node.js 20)	3001	Fastify API + Event Indexer

frontend	Custom (Node.js 20)	3000	Next.js 14 aplikacija

Svi servisi imaju healthcheck-ove za praćenje stanja i automatski restart pri padu. Makefile pruža prečice za sve operacije: make up (pokretanje), make test (testiranje), make build (kompajliranje). Tri Docker Compose fajla podržavaju različite konfiguracije: produkcija, razvoj i testiranje. Za raspoređivanje pametnih ugovora na Sepolia testnet koristi se Foundry skripta, a kod se verzionira kroz Git sa CI/CD pipeline-om na GitHub Actions.

4. ARHITEKTURA PAMETNIH UGOVORA

Ovo poglavlje predstavlja jezgro rada – detaljnu analizu arhitekture, implementacije i bezbedonosnih obrazaca pametnih ugovora sistema dGameBet. Sistem se sastoji od tri ugovora: BetFactory (centralni factory), BetMatch (individualni meč) i BetMath (matematička biblioteka), uz dva interfejsa (IBetFactory, IBetMatch).

4.1. Pregled arhitekture – Factory dizajn obrazac

Factory dizajn obrazac omogućava da jedan centralni ugovor (BetFactory) kreira i prati individualne ugovore za svaki meč (BetMatch). Ovaj pristup pruža izolaciju – greška ili eksploatacija u jednom meču ne utiče na ostale. Svaki BetMatch ugovor ima svoju sopstvenu bazu sredstava i nezavisno stanje.

Tabela 3. Pregled pametnih ugovora sistema

Ugovor	Linije koda	Namena	Deployment
BetFactory	214	Kreiranje mečeva, registar, rejting organizatora	Jedna instanca
BetMatch	279	Logika klađenja, isplate, refund	Jedan po meču
BetMath	44	Pure matematička biblioteka za proračune	Linkovan
IBetFactory	51	Interfejs za BetFactory	–
IBetMatch	61	Interfejs za BetMatch (enumi, eventi, errori)	–

4.2. BetFactory ugovor

BetFactory je centralni ugovor koji služi kao tačka ulaska u sistem. Svaki korisnik koji želi da organizuje klađenje poziva createMatch() funkciju koja validira ulazne parametre, raspoređuje novi BetMatch ugovor i registruje meč u internom nizu.

```
contract BetFactory is IBetFactory {
    // ============================================================
    // State Variables
    // ============================================================

    /// @notice Array of all created matches
    MatchInfo[] private _matches;

    /// @notice Mapping of organizer address to their match indices
    mapping(address => uint256[]) public organizerMatches;

    /// @notice Rating information per organizer
    struct RatingInfo {
        uint256 totalRating;
        uint256 ratingCount;
    }

    mapping(address => RatingInfo) public organizerRatings;

    /// @notice Tracks if a user has rated an organizer for a specific match
    mapping(address => mapping(uint256 => bool)) public hasRated;

    /// @notice Tracks if a user has bet on a specific match (for rating eligibility)
    mapping(address => mapping(uint256 => bool)) public hasBet;
```

```
    /// @notice Block number when this factory was deployed
    uint256 public immutable deployBlock;

    // ============================================================
    // Constructor
    // ============================================================

    constructor() {
        deployBlock = block.number;
    }

    // ============================================================
    // External Functions
    // ============================================================

    /// @notice Create a new betting match
    /// @param teamA Name of Team A
    /// @param teamB Name of Team B
    /// @param matchStartTime Unix timestamp when match starts
    /// @param _betAmount Fixed bet amount in wei
    /// @return matchContract Address of the deployed match contract
    function   createMatch(string   calldata   teamA,   string   calldata   teamB,   uint256
matchStartTime, uint256 _betAmount)
        external
        returns (address matchContract)
    {
        if (bytes(teamA).length == 0 || bytes(teamB).length == 0) {
            revert InvalidTeamName();
        }
        if (matchStartTime <= block.timestamp) {
            revert InvalidStartTime();
        }
        if (_betAmount == 0) {
            revert InvalidBetAmount();
        }

        uint256 matchId = _matches.length;
        BetMatch   newMatch   =   new   BetMatch(address(this),   msg.sender,   teamA,   teamB,
matchStartTime, _betAmount, matchId);

        matchContract = address(newMatch);

        _matches.push(
            MatchInfo({
                matchContract: matchContract,
                organizer: msg.sender,
                teamA: teamA,
                teamB: teamB,
                matchStartTime: matchStartTime,
                betAmount: _betAmount,
                isCompleted: false
            })
        );

        organizerMatches[msg.sender].push(matchId);

        emit MatchCreated(matchId, matchContract, msg.sender, teamA, teamB, matchStartTime,
_betAmount);
    }
```

Listing 4.1. BetFactory – state varijable i createMatch() funkcija

Funkcija createMatch() sprovodi tri validacije pre kreiranja meča: imena timova ne smeju biti prazna (InvalidTeamName), vreme početka mora biti u budućnosti (InvalidStartTime), i iznos opklade mora biti veći od nule (InvalidBetAmount). Nakon validacije, kreira se novi BetMatch ugovor sa svim parametrima i emituje se MatchCreated event.

```
/// @notice Rate an organizer after match completion
    /// @param matchId The match ID
```

```
    /// @param rating Rating from 1 to 5
    function rateOrganizer(uint256 matchId, uint8 rating) external {
        if (matchId >= _matches.length) revert MatchNotFound();
        if (rating < 1 || rating > 5) revert InvalidRating();

        MatchInfo storage matchInfo = _matches[matchId];

        if (!matchInfo.isCompleted) revert MatchNotCompleted();
        if (!hasBet[msg.sender][matchId]) revert NotABettor();
        if (hasRated[msg.sender][matchId]) revert AlreadyRated();

        hasRated[msg.sender][matchId] = true;
        organizerRatings[matchInfo.organizer].totalRating += rating;
        organizerRatings[matchInfo.organizer].ratingCount++;

        emit OrganizerRated(matchInfo.organizer, msg.sender, matchId, rating);
    }

    /// @notice Get organizer's average rating
    /// @param organizer Organizer address
    /// @return average Average rating (multiplied by 100 for precision)
    /// @return count Total number of ratings
    function getOrganizerRating(address organizer) external view returns (uint256 average,
uint256 count) {
        RatingInfo storage info = organizerRatings[organizer];
        count = info.ratingCount;
        if (count > 0) {
            average = (info.totalRating * 100) / count;
        }
    }
```

Listing 4.2. BetFactory – rateOrganizer() funkcija za ocenjivanje organizatora

Sistem ocenjivanja omogućava korisnicima koji su učestvovali u klađenju da ocene organizatora nakon završetka meča (ocena 1–5). Prosečna ocena se čuva pomnožena sa 100 radi preciznosti celobrojne aritmetike. Pored ovih ključnih funkcija, BetFactory pruža i view funkcije za filtriranje mečeva (aktivni, završeni, svi) i callback funkcije koje pozivaju BetMatch ugovori za beleženje opklada i završetaka.

4.3. BetMatch ugovor

BetMatch je srce sistema – ugovor koji upravlja celim životnim ciklusom jednog meča za klađenje. Nasleđuje IBetMatch interfejs i OpenZeppelin ReentrancyGuard za zaštitu od reentrancy napada.

4.3.1. Modifikatori za kontrolu pristupa

```
// ============================================================
    // Modifiers
    // ============================================================

    /// @notice Restricts to before match start
    modifier onlyBeforeMatch() {
        if (block.timestamp >= matchStartTime) revert BettingClosed();
        _;
    }

    /// @notice Restricts to after match start
    modifier onlyAfterMatch() {
        if (block.timestamp < matchStartTime) revert MatchNotStarted();
        _;
    }

    /// @notice Restricts to organizer only
    modifier onlyOrganizer() {
        if (msg.sender != organizer) revert OnlyOrganizer();
```

```
        _;
    }

    /// @notice Restricts to pending result
    modifier onlyPending() {
        if (result != MatchResult.Pending) revert ResultAlreadySet();
        _;
    }
```

Listing 4.3. BetMatch – četiri modifikatora za kontrolu pristupa

Četiri modifikatora kontrolišu ko i kada može pozvati funkcije: onlyBeforeMatch dozvoljava poziv samo pre početka meča, onlyAfterMatch samo posle, onlyOrganizer samo organizatoru, i onlyPending samo dok rezultat nije postavljen.

4.3.2. Postavljanje opklade – placeBet()

```
/// @notice Place a bet on a team
    /// @param team Team to bet on (TeamA or TeamB)
    function placeBet(Team team) external payable onlyBeforeMatch {
        if (msg.value != betAmount) revert InvalidBetAmount();

        if (team == Team.TeamA) {
            betsTeamA[msg.sender] += msg.value;
            totalPoolTeamA += msg.value;
        } else {
            betsTeamB[msg.sender] += msg.value;
            totalPoolTeamB += msg.value;
        }

        BetFactory(factory).recordBet(matchId, msg.sender);

        emit BetPlaced(msg.sender, team, msg.value);
    }
```

Listing 4.4. BetMatch – placeBet() funkcija za postavljanje opklade

Funkcija placeBet() prihvata tačan iznos ETH-a definisan pri kreiranju meča i dodeljuje ga pool-u izabranog tima. Poziv factory.recordBet() beleži učešće korisnika za kasnije pravo na ocenjivanje organizatora.

4.3.3. Postavljanje rezultata – setResult()

```
/// @notice Set the match result
    /// @param _result Match result
    function setResult(MatchResult _result) external nonReentrant onlyOrganizer
onlyAfterMatch onlyPending {
        if (_result == MatchResult.Pending) revert InvalidResult();

        result = _result;

        if (_result != MatchResult.Draw) {
            _payOrganizer();
        }

        BetFactory(factory).markMatchCompleted(matchId);

        emit ResultSet(_result);
    }
```

Listing 4.5. BetMatch – setResult() sa nonReentrant zaštitom

Funkcija setResult() koristi tri modifikatora (onlyOrganizer, onlyAfterMatch, onlyPending) i nonReentrant zaštitu. Ako rezultat nije nerešen, poziva se interna funkcija _payOrganizer() koja šalje 5% provizije organizatoru.

4.3.4. Isplata nagrada – claimPrize() i Pull Payment Pattern

```
/// @notice Claim prize for winners
    function claimPrize() external nonReentrant {
        if (result == MatchResult.Pending) revert ResultNotSet();
        if (result == MatchResult.Draw) revert MatchWasDraw();
        if (hasClaimed[msg.sender]) revert AlreadyClaimed();

        uint256 userBet;
        uint256 winningPool;

        if (result == MatchResult.TeamAWins) {
            userBet = betsTeamA[msg.sender];
            winningPool = totalPoolTeamA;
        } else {
            userBet = betsTeamB[msg.sender];
            winningPool = totalPoolTeamB;
        }

        if (userBet == 0) revert NotAWinner();

        uint256 totalPool = totalPoolTeamA + totalPoolTeamB;
        uint256 prizePool = BetMath.calculatePrizePool(totalPool);
        uint256 prize = BetMath.calculatePrize(userBet, winningPool, prizePool);

        hasClaimed[msg.sender] = true;

        (bool success,) = payable(msg.sender).call{value: prize}("");
        if (!success) revert TransferFailed();

        emit PrizeClaimed(msg.sender, prize);
    }
```

Listing 4.6. BetMatch – claimPrize() sa Checks-Effects-Interactions obrascem

Funkcija claimPrize() demonstrira Checks-Effects-Interactions (CEI) obrazac: prvo proverava sve uslove (Checks), zatim menja stanje – hasClaimed = true (Effects), i tek na kraju šalje ETH korisniku (Interactions). Redosled je kritičan za sprečavanje reentrancy napada. Nagrada se računa proporcionalno udelu korisnika u pobedničkom pool-u.

4.3.5. Refund za nerešene mečeve – claimRefund()

```
/// @notice Claim refund for draws
    function claimRefund() external nonReentrant {
        if (result != MatchResult.Draw) revert MatchNotDraw();
        if (hasClaimed[msg.sender]) revert AlreadyClaimed();

        uint256 totalUserBet = betsTeamA[msg.sender] + betsTeamB[msg.sender];
        if (totalUserBet == 0) revert NoBetsToRefund();

        hasClaimed[msg.sender] = true;

        (bool success,) = payable(msg.sender).call{value: totalUserBet}("");
        if (!success) revert TransferFailed();

        emit RefundClaimed(msg.sender, totalUserBet);
    }
```

Listing 4.7. BetMatch – claimRefund() za potpuni povrat sredstava pri Draw rezultatu

Kada je rezultat nerešen (Draw), nema pobednika i nema provizije organizatora. Svim učesnicima se vraća tačan iznos koji su uložili, uključujući opklade na oba tima ako se korisnik kladio na oba ishoda.

4.3.6. Dijagram stanja meča

Tabela 4. Dijagram stanja meča u BetMatch ugovoru

Stanje	Uslov prelaza	Dostupne funkcije
KREIRAN	createMatch() pozvan	placeBet()
KLAĐENJE	block.timestamp < matchStartTime	placeBet()
ZATVOREN	block.timestamp >= matchStartTime	setResult() (samo organizator)
ZAVRŠEN	setResult() pozvan	claimPrize() ili claimRefund()

4.3.7. Eventi i custom errori

BetMatch emituje šest evenata (BetPlaced, ResultSet, PrizeClaimed, RefundClaimed, OrganizerPaid, OrganizerPaymentFailed) i definiše trinaest custom error-a u IBetMatch interfejsu. Custom errori su gas-efikasnija alternativa klasičnom require() sa string porukama – umesto čuvanja stringa u bytecode-u, koristi se samo selector od 4 bajta, što smanjuje troškove za ~50% pri svakom revert-u.

4.4. BetMath biblioteka

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;


/// @title BetMath
/// @notice Math utilities for betting calculations
/// @dev All calculations use integer math with proper precision handling
library BetMath {
    /// @notice Organizer fee percentage (5%)
    uint256 public constant ORGANIZER_FEE_PERCENT = 5;


    /// @notice Winner percentage (95%)
    uint256 public constant WINNER_PERCENT = 95;


    /// @notice Base for percentage calculations
    uint256 public constant PERCENT_BASE = 100;


    /// @notice Calculate organizer fee from total pool
    /// @param totalPool Total betting pool in wei
    /// @return fee Organizer's fee in wei
    function calculateOrganizerFee(uint256 totalPool) internal pure returns (uint256 fee) {
        fee = (totalPool * ORGANIZER_FEE_PERCENT) / PERCENT_BASE;
    }


    /// @notice Calculate prize pool after organizer fee
    /// @param totalPool Total betting pool in wei
    /// @return prizePool Pool available for winners in wei
    function calculatePrizePool(uint256 totalPool) internal pure returns (uint256 prizePool)
{
        prizePool = (totalPool * WINNER_PERCENT) / PERCENT_BASE;
    }


    /// @notice Calculate individual prize
    /// @param userBet User's bet amount in wei
    /// @param winningPool Total bets on winning team in wei
    /// @param prizePool Pool available for winners in wei
    /// @return prize User's prize amount in wei
    function calculatePrize(uint256 userBet, uint256 winningPool, uint256 prizePool)
        internal
        pure
        returns (uint256 prize)
    {
        if (winningPool == 0) return 0;
```

```
        prize = (userBet * prizePool) / winningPool;
    }
}
```

Listing 4.8. BetMath – kompletna matematička biblioteka za proračun nagrada

BetMath je pure biblioteka bez stanja – sve funkcije su internal pure. Tri konstante definišu distribuciju: 5% organizatoru, 95% pobednicima. Formula za individualnu nagradu je: nagrada = (ulog × nagradniPool) / pobedničkiPool. Na primeru: pri ukupnom pool-u od 10 ETH, ulogu od 1 ETH na Tim A (pool Tima A = 3 ETH), provizija je 0.5 ETH, nagradni pool je 9.5 ETH, a nagrada korisnika je (1/3) × 9.5 = 3.166 ETH, što predstavlja profit od 2.166 ETH (216.6% ROI). Greška zaokruživanja je ograničena na maksimalno 1 wei.

4.5. Bezbedonosni obrasci

Sistem implementira šest bezbedonosnih obrazaca koji štite sredstva korisnika od poznatih vektora napada na pametne ugovore [6][8].

4.5.1. ReentrancyGuard

OpenZeppelin ReentrancyGuard koristi boolean lock koji sprečava rekurzivni poziv funkcija označenih nonReentrant modifikatorom. U dGameBet-u, sve funkcije koje šalju ETH (setResult, claimPrize, claimRefund) koriste ovaj modifikator.

4.5.2. Pull Payment Pattern

Ugovor nikada sam ne šalje ETH korisnicima. Korisnik mora aktivno pozvati claimPrize() ili claimRefund() da povuče svoja sredstva. Ovo sprečava Denial-of-Service napade gde bi zlonamerni ugovor mogao blokirati isplate ostalih korisnika odbijanjem ETH-a u receive() funkciji.

4.5.3. Checks-Effects-Interactions (CEI)

CEI obrazac definiše strogi redosled operacija: 1) Provere svih uslova, 2) Ažuriranje stanja ugovora (hasClaimed = true), 3) Eksterna interakcija (slanje ETH). Ažuriranjem stanja pre eksternog poziva, sprečava se mogućnost da napadač iskoristi callback za ponovni poziv funkcije sa nepromenjenim stanjem.

4.5.4. Custom Errors

Umesto require("poruka") koji čuva string u bytecode-u, koriste se custom errori poput BettingClosed(), InvalidBetAmount(), AlreadyClaimed(). Ovo smanjuje gas troškove za ~50% pri svakom revert-u jer se kodira samo selector od 4 bajta.

4.5.5. Access Control

Četiri modifikatora kontrolišu pristup: onlyOrganizer ograničava na kreatora meča, onlyBeforeMatch/onlyAfterMatch pružaju vremensku kontrolu, i onlyPending osigurava da rezultat nije već postavljen. Callback funkcije (markMatchCompleted, recordBet) proveravaju da je pozivalac odgovarajući BetMatch ugovor.

4.5.6. Input Validation

Svaki ulazni parametar se proverava on-chain: iznos mora biti tačan, vreme starta mora biti u budućnosti, ime tima ne sme biti prazno, ocena mora biti između 1 i 5. Nevalidni pozivi se odmah odbijaju sa odgovarajućim custom error-om bez trošenja gasa na promenu stanja.

Tabela 5. Šest bezbedonosnih obrazaca implementiranih u dGameBet sistemu

Obrazac	Implementacija	Štiti od
ReentrancyGuard	nonReentrant na setResult, claimPrize, claimRefund	Reentrancy napadi
Pull Payment	Korisnik poziva claim*(), ugovor ne šalje sam	DoS napadi
CEI	hasClaimed=true pre call{value}	Reentrancy, state manipulation
Custom Errors	21 error umesto require stringova	Gas rasipanje
Access Control	4 modifikatora + callback provere	Neovlašćeni pristup
Input Validation	On-chain provera svih parametara	Nevalidni ulazi

5. TESTIRANJE I VERIFIKACIJA

Ovo poglavlje analizira višeslojnu strategiju testiranja pametnih ugovora sa ciljem postizanja 100% pokrivenosti koda i verifikacije matematičke korektnosti sistema.

5.1. Strategija testiranja

Tabela 6. Ciljevi pokrivenosti po komponentama

Komponenta	Ciljna pokrivenost	Framework	Broj testova
Pametni ugovori	100%	Foundry (forge test)	40+ unit, 7 fuzz, 8 integ.
Backend	80%+	Vitest	Unit + Integration
Frontend	80%+	Vitest + Playwright	Unit + E2E

Foundry cheatcode-ovi omogućavaju simulaciju blockchain uslova: vm.prank(addr) menja msg.sender, vm.deal(addr, amt) dodeljuje ETH, vm.warp(time) menja block.timestamp, vm.expectRevert(selector) očekuje revert, i vm.expectEmit() verifikuje emitovanje evenata.

5.2. Unit testovi

```
contract BetMatchTest is Test {
    BetFactory public factory;
    BetMatch public betMatch;

    address public organizer = makeAddr("organizer");
    address public bettor1 = makeAddr("bettor1");
    address public bettor2 = makeAddr("bettor2");
    address public bettor3 = makeAddr("bettor3");

    uint256 public constant BET_AMOUNT = 0.1 ether;
    uint256 public matchStartTime;

    function setUp() public {
        factory = new BetFactory();
        matchStartTime = block.timestamp + 1 days;

        vm.deal(organizer, 10 ether);
        vm.deal(bettor1, 10 ether);
        vm.deal(bettor2, 10 ether);
        vm.deal(bettor3, 10 ether);

        vm.prank(organizer);
        address matchAddr = factory.createMatch("Barcelona", "Real Madrid", matchStartTime,
BET_AMOUNT);
        betMatch = BetMatch(matchAddr);
    }
```

Listing 5.1. setUp() – priprema test okruženja za svaki test

Funkcija setUp() se izvršava pre svakog testa i kreira potpuno čisto okruženje: novi BetFactory ugovor, test adrese sa dodeljenim ETH-om, i novi meč Barcelona vs Real Madrid. Svaki test je potpuno nezavisan. Pozitivni testovi verifikuju uspešno beleženje opklada u pool-ove timova, dok negativni testovi koriste vm.warp() i vm.expectRevert() za proveru da se nevalidni pozivi odbijaju sa odgovarajućim custom error-om.

Tabela 7. Kategorije unit testova za BetMatch

Kategorija	Broj testova	Primeri
placeBet() / setResult()	14	Uspeh oba tima, pogrešan iznos, nakon starta, svi ishodi, kontrola

		pristupa
claimPrize() / claimRefund()	12	Pobednik, gubitnik, dupli claim, draw refund
Edge cases	8+	Organizer payment fail, zero bets, reentrancy, asimetrični pool-ovi

5.3. Fuzz testovi i invarijante

Fuzz testovi koriste Foundry-jev fuzzer koji automatski generiše hiljade nasumičnih ulaza za verifikaciju invarijantnih svojstava sistema. Funkcija bound() ograničava ulaze na smislene opsege. Svaki fuzz test se izvršava sa 10.000+ nasumičnih ulaza.

```
function testFuzz_Invariant_FeeDecomposition(uint256 totalPool) public pure {
        totalPool = bound(totalPool, 0, type(uint128).max);


        uint256 fee = BetMath.calculateOrganizerFee(totalPool);
        uint256 prizePool = BetMath.calculatePrizePool(totalPool);


        assertTrue(fee + prizePool <= totalPool, "fee + prizePool must never exceed
totalPool");


        uint256 dust = totalPool - fee - prizePool;
        assertTrue(dust <= 1, "dust from fee decomposition must be at most 1 wei");
    }
```

Listing 5.2. Fuzz test – Invarijanta 1: Fee dekompozicija

Tabela 8. Sedam invarijantnih svojstava verifikovanih fuzz testovima

#	Invarijanta	Formula	Značenje
1	Fee dekompozicija	fee + prizePool ≤ totalPool, dust ≤ 1 wei	Nikad se ne kreira ETH iz ničega
2	Solventnost nagrada	Σ(prizes) ≤ prizePool	Ukupne nagrade ne prelaze pool
3	ETH solventnost	balance ≥ Σ(unclaimed)	Ugovor uvek ima dovoljno ETH
4	Draw refund	Σ(refunds) = totalPool	Sve se vraća pri nerešenom
5	Konzervacija vrednosti	fee + prizes + dust = totalPool	Nijedan wei se ne gubi
6	Proporcionalnost	veći ulog → veća nagrada	Veći ulog donosi veću nagradu
7	Rejting prosek	avg = Σ(ratings) / count	Prosek se korektno računa

5.4. Integracioni testovi

Osam integracionih testova pokriva kompletne tokove od kraja do kraja: kreiranje meča, postavljanje opklada, postavljanje rezultata, isplatu nagrada i ocenjivanje organizatora. Svaki test verifikuje interakciju između BetFactory i BetMatch ugovora kroz celokupan životni ciklus meča.

Tabela 9. Ključni integracioni workflow scenariji (od ukupno osam)

Scenario	Opis
FullWorkflow_TeamAWins	Kreiranje → Klađenje → TeamA pobeđuje → Isplata nagrade → Ocena organizatora
FullWorkflow_Draw	Kreiranje → Klađenje → Draw → Povrat uloga
FullWorkflow_MultipleBettors	4 učesnika, proporcionalna raspodela nagrada

FullWorkflow_RatingSystem	3 učesnika ocenjuju organizatora → provera proseka

5.5. Bezbedonosni testovi

Tri napadačka ugovora simuliraju realne napade na sistem. Svaki napadački ugovor je implementiran kao Solidity contract koji pokušava da eksploatiše specifičnu ranjivost.

```
contract ReentrancyAttacker {
    BetFactory public immutable factory;
    BetMatch public target;
    bool public attacking;


    constructor(BetFactory _factory) { factory = _factory; }
    function createMatch(string calldata a, string calldata b, uint256 t, uint256 amt)
external returns (address) {
        return factory.createMatch(a, b, t, amt);
    }
    function setTarget(BetMatch _target) external { target = _target; }
    function setResult(BetMatch m, IBetMatch.MatchResult r) external {
        m.setResult(r);
    }
    function attack() external { attacking = true; }
    receive() external payable {
        if (attacking) {
            attacking = false;
            target.claimPrize();
        }
    }
}
```

Listing 5.3. ReentrancyAttacker – napadački ugovor koji pokušava reentrancy napad

ReentrancyAttacker pokušava da pozove claimPrize() unutar receive() callback-a kada prima ETH kao organizator. Test verifikuje da se OrganizerPaymentFailed event emituje, da hasClaimed ostaje false, i da sva sredstva ostaju u ugovoru. Na isti način, RejectETH i RejectingBettor testiraju otpornost sistema na odbijanje ETH transfera.

Tabela 10. Tri napadačka ugovora i rezultati bezbedonosnih testova

Napadački ugovor	Tip napada	Rezultat
ReentrancyAttacker	Poziva claimPrize() u receive() callback-u	Blokiran – ReentrancyGuard + CEI
RejectETH	Odbija ETH transfer u receive()	OrganizerPaymentFailed – sistem nastavlja
RejectingBettor	Kladi se pa odbija ETH na claim	TransferFailed – ostali nisu pogođeni

6. BACKEND ARHITEKTURA

6.1. Fastify REST API

Backend koristi Fastify web framework sa TypeScript-om za pružanje REST API-ja i WebSocket konekcija. API pruža tri grupe endpointa:

- /api/v1/matches – lista mečeva, filtriranje po statusu, detalji
- /api/v1/users/:address/bets – opklade korisnika, statistika
- /api/v1/organizers/:address – profil organizatora, rejting, istorija

6.2. Blockchain Event Indexer

Indexer je centralna komponenta backend-a koja sinhronizuje stanje sa blockchain-a u lokalnu PostgreSQL bazu. Koristi viem klijent za osluškivanje šest tipova evenata emitovanih od strane pametnih ugovora.

```
import { parseAbiItem, type Log } from 'viem';
import { publicClient } from '../config/blockchain.js';
import { redisPub } from '../config/redis.js';
import { matchService } from '../services/match-service.js';
import { betService } from '../services/bet-service.js';
import { organizerService } from '../services/organizer-service.js';
import { config } from '../config/index.js';
import { db } from '../config/database.js';
import { syncState } from '../db/schema.js';
import { eq } from 'drizzle-orm';

const MATCH_CREATED_EVENT = parseAbiItem(
  'event MatchCreated(uint256 indexed matchId, address indexed matchContract, address
indexed organizer, string teamA, string teamB, uint256 matchStartTime, uint256 betAmount)'
);

const BET_PLACED_EVENT = parseAbiItem(
  'event BetPlaced(address indexed bettor, uint8 team, uint256 amount)'
);

const RESULT_SET_EVENT = parseAbiItem(
  'event ResultSet(uint8 result)'
);

const PRIZE_CLAIMED_EVENT = parseAbiItem(
  'event PrizeClaimed(address indexed winner, uint256 amount)'
);

const REFUND_CLAIMED_EVENT = parseAbiItem(
  'event RefundClaimed(address indexed bettor, uint256 amount)'
);

const ORGANIZER_RATED_EVENT = parseAbiItem(
  'event OrganizerRated(address indexed organizer, address indexed rater, uint256 indexed
matchId, uint8 rating)'
);
```

Listing 6.1. Blockchain Indexer – definicije evenata koji se indeksiraju

Pipeline obrade podataka: 1) Blockchain emituje event, 2) Indexer osluškuje blokove i filtrira logove, 3) PostgreSQL čuva obrađene podatke, 4) Redis keširа i objavljuje promene, 5) WebSocket šalje ažuriranja klijentima.

6.3. Baza podataka

Drizzle ORM pruža type-safe pristup PostgreSQL bazi sa pet tabela. Schema definiše relacije između tabela (bets referencira matches, ratings referencira organizers) i koristi UUID primarne ključeve sa automatskim generisanjem.

Tabela 11. Schema baze podataka sa pet tabela

Tabela	Ključna polja	Namena
matches	contract_address, organizer, team_a, team_b, result	Indeksirani mečevi
bets	match_id, bettor, team, amount, tx_hash	Individualne opklade
organizers	address, total_matches, average_rating	Profili organizatora
ratings	organizer, rater, match_id, rating	Ocene organizatora
sync_state	last_block_number	Stanje sinhronizacije indeksera

6.4. Redis keš i Pub/Sub

Redis služi dvostruku ulogu: 1) Keširanje aktivnih mečeva sa 30-sekundnim TTL-om za brze odgovore na česte upite, i 2) Pub/Sub kanal (dgamebet:events) za distribuciju promena svim backend instancama i WebSocket klijentima u realnom vremenu, bez potrebe za osvežavanjem stranice.

7. FRONTEND APLIKACIJA

7.1. Next.js 14 aplikacija

Frontend je izgrađen sa Next.js 14 koristeći App Router za rutiranje i Server-Side Rendering za poboljšane performanse. Vizuelni dizajn koristi Tailwind CSS sa tamnom temom i responsive layoutom.

7.2. Integracija sa blockchain-om

Wagmi hooks [10] omogućavaju deklarativnu interakciju sa blockchain-om: useReadContract za čitanje stanja ugovora, useWriteContract za slanje transakcija, i useWaitForTransactionReceipt za praćenje potvrde. RainbowKit pruža UI za konekciju novčanika (MetaMask, WalletConnect). Konfiguracija podržava Sepolia testnet i lokalni Hardhat čvor.

```
'use client';


import { useState, useEffect } from 'react';
import { useWriteContract, useWaitForTransactionReceipt } from 'wagmi';
import { toast } from 'sonner';
import { Button } from '@/components/ui/Button';
import { Modal } from '@/components/ui/Modal';
import { BET_MATCH_ABI } from '@/lib/contracts';
import { formatEth } from '@/lib/utils';
import { Team } from '@/types/match';


interface BetFormProps {
  matchAddress: `0x${string}`;
  teamA: string;
  teamB: string;
  betAmount: bigint;
  disabled?: boolean;
}


export function BetForm({ matchAddress, teamA, teamB, betAmount, disabled }: BetFormProps) {
  const [selectedTeam, setSelectedTeam] = useState<Team | null>(null);
  const [showConfirm, setShowConfirm] = useState(false);


  const { writeContract, data: hash, isPending, error, reset } = useWriteContract();
  const { isLoading: isConfirming, isSuccess } = useWaitForTransactionReceipt({ hash });


  useEffect(() => {
    if (error) {
      toast.error(
        error.message.includes('User rejected')
          ? 'Transaction rejected by user.'
          : 'Transaction failed. Please try again.'
      );
      reset();
    }
  }, [error, reset]);


  function handleBet() {
    if (selectedTeam === null) return;
    writeContract({
      address: matchAddress,
      abi: BET_MATCH_ABI,
      functionName: 'placeBet',
      args: [selectedTeam],
      value: betAmount,
    });
    setShowConfirm(false);
  }
```

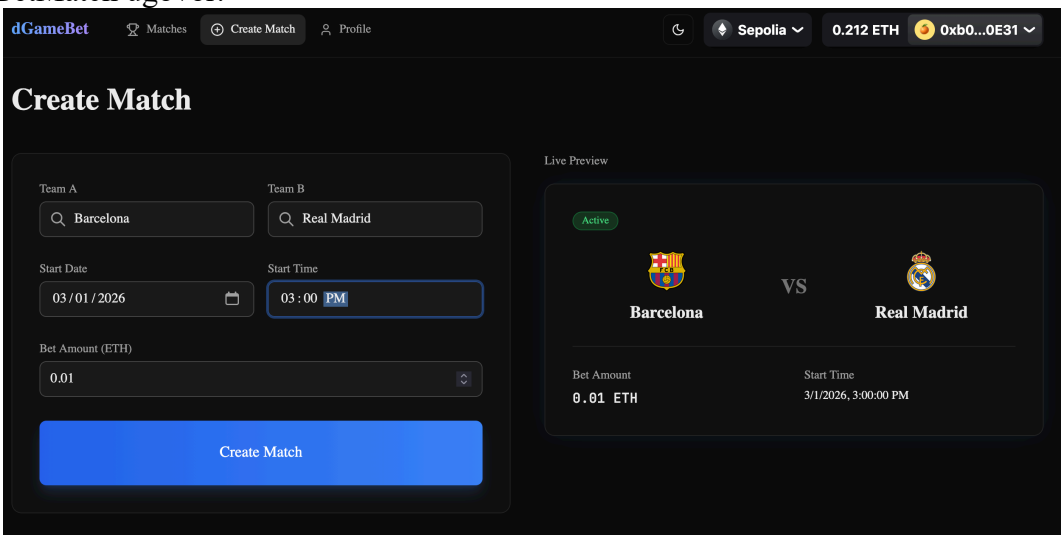
Listing 7.1. BetForm komponenta – UI za postavljanje opklade sa wagmi integracijom

7.3. Korisničko iskustvo – demonstracija

Aplikacija podržava dva glavna korisnika: organizatora koji kreira mečeve i postavlja rezultate, i kladilac koji se kladi i preuzima nagrade.

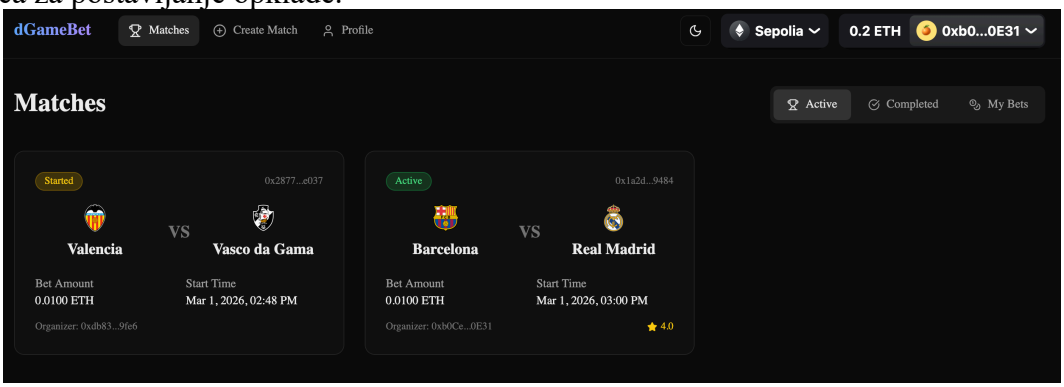
7.3.1. Kreiranje meča i klađenje

Organizator popunjava formu za kreiranje meča birajući timove kroz auto-complete pretragu, postavljajući vreme početka i fiksni iznos opklade. Live preview kartica prikazuje kako će meč izgledati pre slanja transakcije. Nakon potvrde kroz MetaMask, deploy-uje se novi BetMatch ugovor.

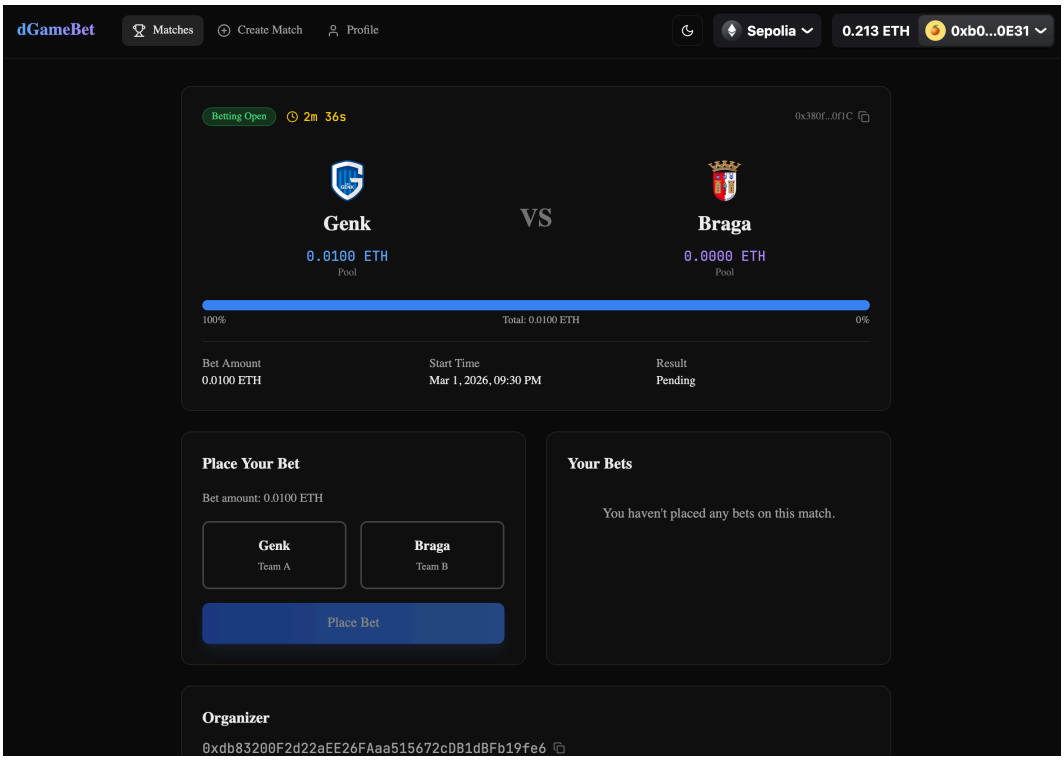


Slika 7.1. Kreiranje meča – forma sa auto-complete pretragom timova i live preview karticom

Kreirani mečevi se prikazuju na stranici aktivnih mečeva sa vizuelnim pool barovima koji pokazuju raspodelu opklada u realnom vremenu. Klikom na meč otvara se detaljna stranica za postavljanje opklade.



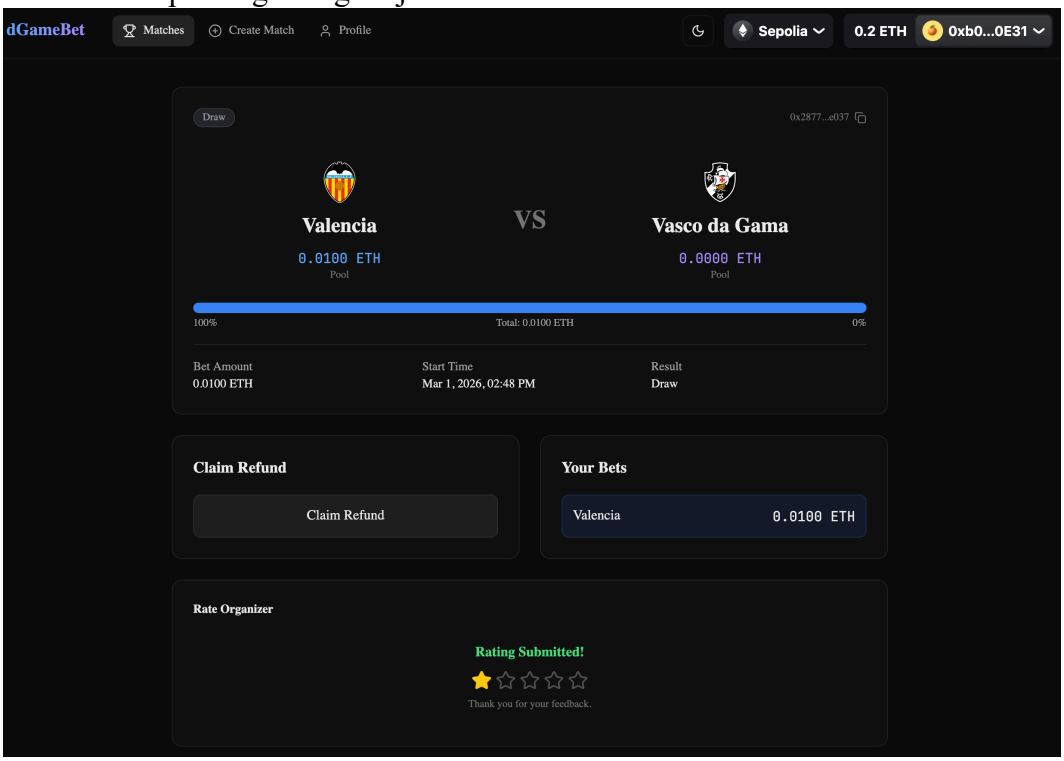
Slika 7.2. Lista aktivnih mečeva sa pool barovima i vremenskim odbrojavanjem



Slika 7.3. Detaljna stranica meča – izbor tima i postavljanje opklade

7.3.2. Rezultati i Pull Payment u praksi

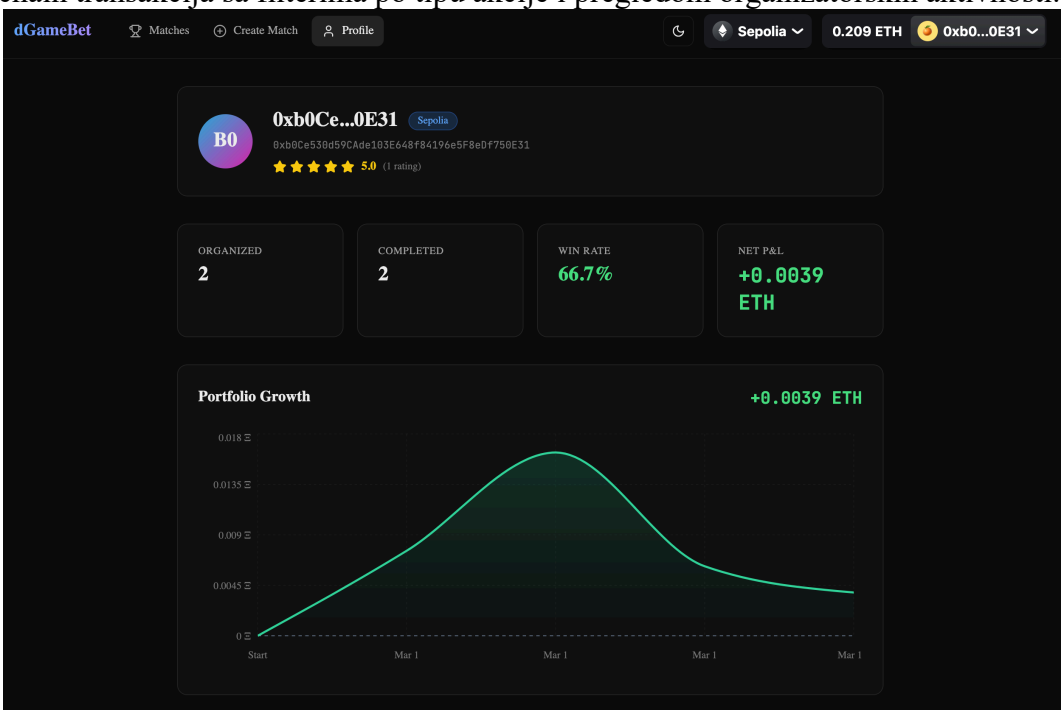
Nakon završetka meča, organizator postavlja rezultat. U slučaju nerešenog ishoda, korisnici pozivaju claimRefund() za potpuni povrat sredstava – praktična demonstracija Pull Payment obrasca opisanog u Poglavlju 4.



Slika 7.4. Draw rezultat meča – Claim Refund dugme ilustruje Pull Payment u praksi

7.3.3. Portfolio i analitika

Korisnički profil prikazuje statistiku: ukupni P&L (profit & loss) u ETH, Win Rate procenat, broj opklada i Portfolio Growth grafikon. Profil takođe sadrži kompletnu istoriju blockchain transakcija sa filterima po tipu akcije i pregledom organizatorskih aktivnosti.



Slika 7.5. Profil sa P&L statistikom i Portfolio Growth grafikonom

8. ZAKLJUČAK I BUDUĆI RAD

8.1. Ostvareni rezultati

Projekat dGameBet uspešno demonstrira primenu blockchain tehnologije u domenu decentralizovanog klađenja. Ključni rezultati uključuju:

- Sistem pametnih ugovora sa Factory obrascem i 100% pokrivenošću testovima
- Više bezbedonosnih obrazaca verifikovanih kroz tri napadačka ugovora
- Moderan web interfejs sa MetaMask integracijom i portfolio analitikom
- Event-driven backend sa blockchain indekserom i kontejnerizovano okruženje

8.2. Ograničenja

- Organizator ručno postavlja rezultat – podložno grešci ili zloupotrebi
- Fiksni iznos opklade i podrška samo za fudbal
- Raspoređeno na Sepolia testnet, nije na mainnet-u

8.3. Primena u realnom svetu

Iako je dGameBet razvijen kao akademski projekat na Sepolia testnet-u, arhitektura i obrasci koji su implementirani imaju direktnu primenu u brojnim realnim scenarijima. U nastavku su predstavljeni ključni slučajevi upotrebe koji ilustruju praktičnu vrednost decentralizovanog klađenja.

8.3.1. Decentralizovano sportsko klađenje

Tradicionalne kladionice posluju po netransparentnom modelu gde korisnici ne mogu da verifikuju kako se kvote formiraju, koliki je stvarni fond nagrada, niti da li je isplata izvršena pravedno. Prema izveštajima FATF-a (Financial Action Task Force) i Transparency International-a, industrija klađenja je jedna od najranjivijih na manipulaciju i pranje novca – FATF je identifikovao kazina i sektor igara na sreću kao visoko rizične za ML/TF (Money Laundering / Terrorist Financing) [11]. dGameBet model rešava ove probleme: svi ulozi su vidljivi na blockchain-u, formula za distribuciju nagrada je javno dostupna u izvornom kodu pametnog ugovora, a isplata se izvršava automatski bez mogućnosti ljudske intervencije. Ovo predstavlja fundamentalni pomak od modela 'veruj kladionici' ka modelu 'verifikuj sam na blockchain-u'.

8.3.2. Peer-to-peer klađenje bez posrednika

Factory obrazac implementiran u dGameBet-u omogućava svakome da postane organizator meča – bez potrebe za licencom, infrastrukturom ili kapitalom kladionice. Ovo otvara mogućnost peer-to-peer klađenja gde korisnici direktno kreiraju tržišta za sportske događaje. Platforme poput SBET Protocol-a i OpenTote-a na Solana mreži već demonstriraju ovaj model u praksi, nudeći provizije od 1.5–3% naspram tradicionalnih 5–10% marži kladionica. U regionima sa restriktivnim zakonima o kockanju ili nerazvijenoj infrastrukturi za online klađenje, ovakav model pruža pristupačnu alternativu. Svako sa Ethereum novčanikom može učestvovati, čime se eliminiše geografska i regulatorna diskriminacija prisutna u tradicionalnim platformama.

8.3.3. DAO-upravljane kladioničke zajednice

Sistem organizatora sa reputacijom (rateOrganizer funkcija) predstavlja temelj za složenije modele upravljanja. U realnom scenariju, organizatori mogu biti zamenjeni DAO (Decentralized Autonomous Organization) strukturama gde zajednica glasanjem odlučuje o kreiranju mečeva, postavljanju parametara i rešavanju sporova. Koncept futarchy-ja, koji je predložio ekonomista Robin Hanson ('glasaj o vrednostima, kladi se na uverenja'), već je implementiran u projektima poput MetaDAO na Solana mreži, gde učesnici ulažu kapital na ishode predloga umesto tradicionalnog glasanja. Ovakav model eliminiše potrebu za centralnim autoritetom i omogućava samoupravljačke kladioničke zajednice gde korisnici imaju direktan uticaj na pravila i operacije platforme.

8.3.4. Transparentnost i zaštita korisnika

Jedna od najznačajnijih prednosti blockchain klađenja je inherentna zaštita korisnika. U tradicionalnom modelu, korisnik nema garanciju da kladionica poseduje dovoljno sredstava za isplatu svih dobitaka – problem poznat kao rizik nesolventnosti. U dGameBet sistemu, sva sredstva su zaključana u pametnom ugovoru i mogu se povući isključivo prema unapred definisanim pravilima. Pull Payment obrazac garantuje da korisnik uvek može preuzeti svoja sredstva, dok CEI obrazac štiti od manipulacije stanjem. Ovo eliminiše potrebu za regulatornim telima koja nadziru solventnost operatora, jer je solventnost matematički garantovana kodom pametnog ugovora.

8.3.5. Integracija sa DeFi ekosistemom

dGameBet arhitektura je kompatibilna sa širim DeFi (Decentralized Finance) ekosistemom. U budućim implementacijama, neaktivna sredstva u pool-ovima mečeva koji čekaju rezultat mogla bi se plasirati u yield protokole (kao što su Aave ili Compound), generišući dodatni prinos za učesnike – model koji već primenjuje platforma Divvy.bet na Solana mreži, gde provajderi likvidnosti zarađuju prinos na kladioničkim pool-ovima. Takođe, ERC-20 tokeni bi mogli zameniti ETH kao sredstvo klađenja, omogućavajući integraciju sa stablecoin-ima (USDC, DAI) za eliminaciju volatilnosti kriptovaluta. Tokenizacija pozicija klađenja kroz NFT-ove već je implementirana na platformama poput JuicyBet-a (Azuro protokol), gde korisnici mogu trgovati opkladama kao NFT tokenima na sekundarnom tržištu pre završetka meča.

8.3.6. Regulatorna usklađenost i revizija

Blockchain prirodno pruža kompletnu revizijsku trasu (audit trail) za svaku transakciju. Regulatorna tela mogu u realnom vremenu pratiti tokove sredstava, verifikovati pravednost isplata i detektovati sumnjive obrasce ponašanja – bez oslanjanja na izveštaje samih operatora. EU regulativa MiCA (Markets in Crypto-Assets Regulation), implementirana u fazama tokom 2024–2025. godine, uvodi obavezu registracije kripto-servis provajdera (CASP) i usklađenosti sa AML/CFT direktivama (EU 2015/849) [12]. Ova transparentnost značajno pojednostavljuje proces regulatorne revizije – prema EGBA (European Gaming and Betting Association), operatori su podneli gotovo 13.000 prijava sumnjivih aktivnosti samo u 2021. godini, što ilustruje obim potrebe za automatizovanim nadzorom. Međutim, činjenica da blockchain transakcije ne otkrivaju direktno identitet korisnika predstavlja izazov za primenu KYC (Know Your Customer) procedura, te buduće implementacije zahtevaju balans između zaštite privatnosti i ispunjavanja regulatornih

obaveza. dGameBet demonstrira kako tehnologija može proaktivno odgovoriti na zahteve regulatora kroz ugrađenu transparentnost umesto naknadnog izveštavanja.

Tabela 12. Realni slučajevi upotrebe i doprinos dGameBet sistema

Slučaj upotrebe	Ključna prednost	dGameBet doprinos
Sportsko klađenje	Potpuna transparentnost fondova	On-chain pool-ovi i automatska distribucija
P2P klađenje	Eliminacija posrednika	Factory obrazac – svako može biti organizator
DAO kladionice	Decentralizovano upravljanje	Sistem reputacije organizatora
Zaštita korisnika	Garantovana solventnost	Pull Payment + sredstva zaključana u ugovoru
DeFi integracija	Dodatni prinos i likvidnost	Kompatibilna EVM arhitektura
Regulatorna revizija	Automatski audit trail	Javni blockchain zapisi svih transakcija

8.4. Budući pravci razvoja

- Chainlink Oracle za automatsko postavljanje rezultata
- Layer 2 deployment (Optimism/Arbitrum) za niže transakcione troškove
- Višesportska podrška i mobilna aplikacija

LITERATURA

[1] Buterin, V., "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," White Paper, 2014.

[2] Szabo, N., "Formalizing and Securing Relationships on Public Networks," First Monday, vol. 2, no. 9, 1997.

[3] Nakamoto, S., "Bitcoin: A Peer-to-Peer Electronic Cash System," White Paper, 2008.

[4] Wood, G., "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Yellow Paper, 2014.

[5] A. Antonopoulos, G. Wood, Mastering Ethereum: Building Smart Contracts and DApps, O'Reilly Media, Sebastopol, 2018.

[6] Atzei, N.; Bartoletti, M.; Cimoli, T., "A Survey of Attacks on Ethereum Smart Contracts (SoK)," in POST, 2017.

[7] OpenZeppelin, "Contracts Library," https://docs.openzeppelin.com/contracts (pristupljeno 1. marta 2026.).

[8] Consensys, "Ethereum Smart Contract Best Practices," https://consensys.github.io/smart-contract-best-practices (pristupljeno 1. marta 2026.).

[9] Foundry Book, https://book.getfoundry.sh (pristupljeno 1. marta 2026.).

[10] wagmi, "React Hooks for Ethereum," https://wagmi.sh (pristupljeno 1. marta 2026.).

[11] FATF, "Vulnerabilities of Casinos and Gaming Sector," FATF Report, https://www.fatf-gafi.org/en/publications/Methodsandtrends/Vulnerabilitiesofcasinosandgamingsector.html (pristupljeno 5. marta 2026.).

[12] European Parliament, "Regulation (EU) 2023/1114 on Markets in Crypto-Assets (MiCA)," Official Journal of the European Union, L 150, 9. jun 2023.